PATTERNWISE

An Interactive Software Design Patterns Visual Guide

CAPSTONE PROJECT SUBMISSION REPORT

Student Name	Mahalakshmi V
Year / Class	2nd Year, CSE - B
Project Type	Educational web application
Technology	Python, Flask, HTML, CSS, JavaScript

Prepared as a student project report. Update college, department guide, register number, and submission date if your institution requires them.

ABSTRACT

PatternWise is an educational web application designed to help students understand common software design patterns through clear explanations, Python examples, diagrams, and quiz-based revision. The project groups eight patterns into creational, structural, and behavioral categories. It provides a central place to explore the purpose of each pattern, inspect example implementations, and practice identifying patterns through learning activities. The project is built with Python and Flask for application logic and routing, and HTML, CSS, and JavaScript for the user interface and interactive behavior. Supporting materials include diagrams, tests, research notes, a quiz with answer key, and project documentation. The application is intended for learning and demonstration; it is not a replacement for formal software architecture review.

1. INTRODUCTION

Software design patterns are reusable approaches to recurring design problems. Learning them can be challenging when the concepts are presented only as definitions, so PatternWise combines short explanations with code and visual learning materials. The application is intended for students and beginners who want to revise design patterns and see how they can be represented in Python.

2. PROBLEM STATEMENT

Beginners may find it difficult to distinguish software design patterns, understand when to use them, and connect their structure to practical code. Learning resources are often spread across notes, code samples, and diagrams. PatternWise addresses this learning challenge by organizing selected patterns into one interactive educational website.

3. OBJECTIVES

- Explain eight common software design patterns in beginner-friendly language.
- Show Python implementations in separate, organized modules.
- Provide diagrams and examples to support conceptual understanding.
- Offer quiz questions and feedback for revision.
- Include automated tests and documentation to support verification and presentation.

4. SCOPE OF THE PROJECT

The project focuses on eight patterns: Singleton, Factory Method, Builder, Adapter, Decorator, Facade, Strategy, and Observer. It includes a Flask-based website, pattern source modules, frontend assets, diagram source files, quiz materials, test files, and supporting documentation. The core scope is educational demonstration rather than production-scale deployment.

5. DESIGN PATTERNS INCLUDED

Category	Pattern	Purpose / Example focus
Creational	Singleton	Restricts creation to a shared instance; example: shared configuration manager.
Creational	Factory Method	Creates objects through a common creation interface; example: notifications.
Creational	Builder	Builds a complex object step by step; example: computer configuration.

Category	Pattern	Purpose / Example focus
Structural	Adapter	Makes incompatible interfaces work together; example: legacy service adapter.
Structural	Decorator	Adds behavior without changing the original class; example: optional coffee features.
Structural	Facade	Provides a simpler interface to multiple subsystems; example: home theatre.
Behavioral	Strategy	Allows interchangeable behavior or algorithms; example: payment strategies.
Behavioral	Observer	Notifies subscribers when a subject changes; example: notification subscriptions.

6. TECHNOLOGIES USED

Python: used for design-pattern implementations and application-side logic.
Flask: used to serve pages and handle routes.
HTML5: defines the structure of website pages.
CSS3: provides layout, styling, and responsive presentation.
JavaScript: supports client-side interactions and quiz behavior.
pytest / Python test modules: automated checks are included in the project package; the bundled run-all script executes the available project checks.

7. SYSTEM ARCHITECTURE

The application is organized into layers. The Flask entry point receives browser requests and renders templates. Pattern modules contain the individual Python examples, while the catalog organizes pattern information for display. Templates provide page structure, static CSS styles the interface, and JavaScript supports browser-side interactions. Documentation, diagrams, quizzes, and tests are kept in separate folders to make the project easier to maintain.

```
Browser → Flask application (app.py) → pattern catalog / pattern modules → HTML templates and static assets
```

8. DEVELOPMENT METHODOLOGY

1. Requirement planning: selected a focused set of eight patterns and learning features.
2. Implementation: created a separate Python module for each pattern and organized related modules by category.
3. Website development: created Flask routes, HTML templates, CSS styling, and JavaScript interactions.
4. Learning materials: prepared diagram sources, quiz content, answer key, research notes, and demo guidance.
5. Verification: ran the bundled project checks and unit tests in the reported environment.
6. Publishing: initialized Git, committed the project, connected the remote repository, and pushed the main branch to GitHub.

9. TESTING AND VERIFICATION

The project package includes tests for the design-pattern examples and selected application behavior. The run-all output previously reported for this project showed eight pattern example checks passing, an application listing check passing, and 103 unit tests passing, for a total of 10 checks passed and 0 failed in that environment. These are recorded results from the earlier run and should be reproduced on the final submission computer before demonstration. A successful local test run does not by itself verify deployment, accessibility, or production security.

10. HOW TO RUN THE PROJECT

1. Extract the project folder.
2. Open the inner **PatternWise** folder in VS Code.
3. Open Terminal → New Terminal.
4. Install dependencies with: `python -m pip install -r requirements.txt`
5. Start the application with: `python app.py`
6. Open `http://127.0.0.1:5000` in a browser.
7. Keep the terminal running while using the site. Press Ctrl+C in the terminal to stop the server.

11. GITHUB REPOSITORY

The source code was pushed to the GitHub repository:
https://github.com/MahalakshmiV10/PatternWise
The repository contains the project files. A GitHub repository link is not the same as a deployed live website. A separate hosting setup and live URL check are required before claiming that the website is publicly hosted.

12. LIMITATIONS

- The project is intended for educational use and demonstrates selected patterns with simplified examples.
- The examples do not cover every possible implementation or trade-off for each pattern.
- A local Flask development server is not intended for production deployment.
- The live-hosting status must be checked separately; pushing code to GitHub does not automatically deploy a Flask application.
- Test results should be rerun after any code changes and on the final demonstration computer.

13. FUTURE ENHANCEMENTS

Possible future work includes adding more patterns, expanding the question bank, improving accessibility, adding more interactive visualizations, allowing users to export learning progress, and deploying the website to a hosting platform after configuration and verification.

14. CONCLUSION

PatternWise brings explanations, Python examples, diagrams, quizzes, and supporting documentation together in one learning-focused website. By organizing the eight patterns into creational, structural, and behavioral categories, the project aims to help beginners revise core software design concepts and relate them to code. The project includes source code, automated tests, and submission materials that support a repeatable demonstration. Further deployment and final-environment checks should be completed before presenting the project as a live website.

REFERENCES / PROJECT MATERIALS

Project source repository: https://github.com/MahalakshmiV10/PatternWise
Project README and included documentation: project overview, pattern notes, research report, testing report, diagram files, quiz materials, and demo script. External references should be added with exact titles, URLs, and access dates if required by the department's report format.

STUDENT DECLARATION

I declare that this report describes the PatternWise educational software design patterns guide prepared for academic demonstration. Any external material used in the final submission should be acknowledged according to the institution's guidelines.

Student Signature: _____

Name: Mahalakshmi V

Class: 2nd Year, CSE - B

Date: _____

Faculty / Guide Signature: _____